

Fingerprint Algorithm SDK

Fingerprint Algorithm SDK

FingerprintLicenseAPI

- 1.1 Verify License
- 1.2 License by Account
- 1.3 License by Activation Code
- 1.4 LicenceResult

FingerAlgAPI

- 2.1 Initialize Algorithm
- 2.2 Free Algorithm
- 2.3 Calculate Fingerprint Quality
- 2.4 Calculate Fingerprint Area
- 2.5 Calculate NFIQ
- 2.6 Create ANSI (2004) Template
- 2.7 Create ANSI (2009) Template
- 2.8 Create ISO (2005) Template
- 2.9 Create ISO (2011) Template
- 2.10 – 2.13 Template Matching (1:1 Verification)
- 2.14 – 2.17 Template Searching (1:N Identification)
- 2.18 – 2.21 Image Compression / Decompression
- 2.22 – 2.27 FIR Compression / Decompression (ISO 2005 & 2011)
- 2.28 – 2.30 Image Format Conversion
- 2.31 Draw Template Minutiae
- 2.32 Create Template with Finger Position
- 2.33 Merge Multiple Features into One Record
- 2.34 Compare Templates with Multiple Features and Score

FingerprintLicenseAPI

1.1 Verify License

```
/**
 * Checks whether the license file is valid.
 *
 * @param context      Android application context
 * @param targetFilePath Path to the license file
 * @return true if the license is valid, false otherwise
 */
boolean checkLicense(Context context, String targetFilePath);
```

1.2 License by Account

```

/**
 * Obtains the license file using account credentials.
 *
 * @param context      Android application context
 * @param targetFilePath Path to save the license file
 * @param account      Account name
 * @param key          Account key
 * @return LicenceResult containing the result and license information
 * @see #licenseByActivationCode(Context, String, String)
 */
LicenceResult licenseByAccount(Context context, String targetFilePath, String account,
String key);

```

1.3 License by Activation Code

```

/**
 * Obtains the license file using an activation code.
 *
 * @param context      Android application context
 * @param targetFilePath Path to save the license file
 * @param code         Activation code
 * @return LicenceResult containing the result and license information
 * @see #licenseByAccount(Context, String, String, String)
 */
LicenceResult licenseByActivationCode(Context context, String targetFilePath, String
code);

```

1.4 LicenceResult

```

class LicenceResult {
    /** 0 indicates success, otherwise failure */
    public final int code;

    /** Error message if any */
    public final String errorMsg;

    /** Licensed account */
    public final String account;

    /** Absolute path of the license file */
    public final String filePath;
}

```

FingerAlgAPI

2.1 Initialize Algorithm

```

/**
 * Initializes the fingerprint algorithm.
 *
 * @param context      Android application context
 * @param licensePath Path to the license file
 * @return 0 if successful, otherwise an error code
 */
int init(Context context, String licensePath);

```

2.2 Free Algorithm

```

/**
 * Releases resources allocated by the fingerprint algorithm.
 *
 * @return 0 if successful, otherwise an error code
 */
int free();

```

2.3 Calculate Fingerprint Quality

```

/**
 * Returns the quality score of a fingerprint image.
 * The score follows ANSI/INCITS 381 and ISO/IEC 19794-4 standards.
 *
 * @param rawImage Fingerprint image data
 * @param width     Image width
 * @param height    Image height
 * @return Quality score (0~100), negative for error
 */
int getImageQuality(byte[] rawImage, int width, int height);

```

2.4 Calculate Fingerprint Area

```

/**
 * Returns the area score of a fingerprint image.
 *
 * @param rawImage Fingerprint image data
 * @param width     Image width
 * @param height    Image height
 * @return Area score (0~100), negative for error
 */
int getFingerprintArea(byte[] rawImage, int width, int height);

```

2.5 Calculate NFIQ

```

/**
 * Returns the NFIQ (1~5) of a fingerprint image.
 *
 * @param rawImage Fingerprint image data
 * @param width    Image width
 * @param height   Image height
 * @return NFIQ score (1~5), negative for error
 */
int getNFIQ(byte[] rawImage, int width, int height);

```

2.6 Create ANSI (2004) Template

```

/**
 * Creates an ANSI/INCITS 378-2004 template from a raw fingerprint image.
 *
 * @param rawImage      Raw fingerprint image
 * @param width          Image width
 * @param height         Image height
 * @param minMinutiaeCount Minimum number of minutiae points required
 * @param template       Output template (max 1024 bytes)
 * @param extension      Add extended data for faster search
 * @param templateLength Output template length
 * @return 0 if successful, negative for error
 */
int createTemplateANSI(byte[] rawImage, int width, int height, int minMinutiaeCount,
byte[] template, boolean extension, int[] templateLength);
int createTemplateANSI(byte[] rawImage, int width, int height, byte[] template, boolean
extension, int[] templateLength);
int createTemplateANSI(byte[] rawImage, int width, int height, byte[] template);

```

2.7 Create ANSI (2009) Template

```

int createTemplateANSI2009(byte[] rawImage, int width, int height, byte[] template);
int createTemplateANSI2009(byte[] rawImage, int width, int height, int minMinutiaeCount,
byte[] template, int[] length);

```

2.8 Create ISO (2005) Template

```

int createTemplateISO(byte[] rawImage, int width, int height, int minMinutiaeCount, byte[]
template, boolean extension, int[] templateLength);
int createTemplateISO(byte[] rawImage, int width, int height, byte[] template, boolean
extension, int[] templateLength);
int createTemplateISO(byte[] rawImage, int width, int height, byte[] template);

```

2.9 Create ISO (2011) Template

```
int createTemplateISO2011(byte[] rawImage, int width, int height, int minMinutiaeCount,
byte[] template, boolean extension, int[] templateLength);
int createTemplateISO2011(byte[] rawImage, int width, int height, byte[] template, boolean
extension, int[] templateLength);
int createTemplateISO2011(byte[] rawImage, int width, int height, byte[] template);
```

2.10 - 2.13 Template Matching (1:1 Verification)

```
int compareTemplatesANSI(byte[] template1, byte[] template2);
int compareTemplatesANSI2009(byte[] template1, byte[] template2);
int compareTemplatesISO(byte[] template1, byte[] template2);
int compareTemplatesISO2011(byte[] template1, byte[] template2);
```

2.14 - 2.17 Template Searching (1:N Identification)

```
int searchTemplatesANSI(byte[] template, int numberOfDbTemplates, byte[] dbTemplates);
int searchTemplatesANSI2009(byte[] template, int numberOfDbTemplates, byte[] dbTemplates);
int searchTemplatesISO(byte[] template, int numberOfDbTemplates, byte[] dbTemplates);
int searchTemplatesISO2011(byte[] template, int numberOfDbTemplates, byte[] dbTemplates);
```

2.18 - 2.21 Image Compression / Decompression

```
int compressWSQ(byte[] originImage, int width, int height, int dpi, int compressRatio,
byte[] output, int[] outputLength);
int decompressWSQ(byte[] wsqData, int wsqLen, byte[] rawImage, int[] width, int[] height,
int[] dpi);
int compressJPEG2000(byte[] rawImage, int width, int height, int dpi, int compressRatio,
byte[] output, int[] outputLength);
int decompressJPEG2000(byte[] jpegData, int jpegLen, byte[] rawImage, int[] width, int[]
height, int[] dpi);
```

2.22 - 2.27 FIR Compression / Decompression (ISO 2005 & 2011)

```
int makeFIR(byte[] rawBuf, int width, int height, int compressType, int compressRatio,
byte[] firBuffer, int[] firSize);
int makeFIRBySize(byte[] rawBuf, int width, int height, int compressType, int minSize, int
maxSize, byte[] firBuffer, int[] firSize);
int unmakeFIR(byte[] firBuffer, byte[] rawBuffer, int[] width, int[] height, int[] dpi,
int[] type);

int makeFIR2011(byte[] rawBuf, int width, int height, int compressType, int compressRatio,
byte[] firBuffer, int[] firSize);
int makeFIRBySize2011(byte[] rawBuf, int width, int height, int compressType, int minSize,
int maxSize, byte[] firBuffer, int[] firSize);
int unmakeFIR2011(byte[] firBuffer, byte[] rawBuffer, int[] width, int[] height, int[]
dpi, int[] type);
```

2.28 - 2.30 Image Format Conversion

```
int convertBMPToRaw(byte[] bmpData, int width, int height, byte[] rawImage);
int convertRawToBMP(byte[] rawImage, int width, int height, byte[] bmpData);
int convertRawToBMPRGB(byte[] rawImage, int width, int height, byte[] bmpData);
```

2.31 Draw Template Minutiae

```
int drawPointImg(byte[] imgData, int width, int height);
```

2.32 Create Template with Finger Position

```
/**
 * Creates a fingerprint template including finger position information.
 *
 * @param templateFormat    Template format:
 *                          2 - ANSI INCITS 378-2004
 *                          3 - ANSI INCITS 378-2009
 *                          4 - ISO/IEC 19794-2:2005
 *                          5 - ISO/IEC 19794-2:2011
 * @param rawImage          Raw fingerprint image
 * @param width             Image width
 * @param height            Image height
 * @param template          Output template (max 1024 bytes)
 * @param fingerPosition    Finger position code (1~10)
 * @return 0 if successful, negative number for error code
 */
int createTemplateWithPosition(int templateFormat, byte[] rawImage, int width, int height,
byte[] template, int fingerPosition);

/**
 * Creates a fingerprint template including finger position information, with optional
 * extended data.
 *
 * @param templateFormat    Template format (same codes as above)
 * @param rawImage          Raw fingerprint image
 * @param width             Image width
 * @param height            Image height
 * @param template          Output template (max 1024 bytes)
 * @param extension         Whether to add extended data for faster search
 * @param fingerPosition    Finger position code (1~10)
 * @return 0 if successful, negative number for error code
 */
int createTemplateWithPosition(int templateFormat, byte[] rawImage, int width, int height,
byte[] template, boolean extension, int fingerPosition);
```

2.33 Merge Multiple Features into One Record

```

/**
 * Merges multiple fingerprint feature records into a single feature record.
 *
 * @param templateFormat  Template format (same codes as above)
 * @param iFeatureList    Input feature list
 * @param iFeatureNum     Number of features in the list
 * @param oFeatureRecord  Output merged feature record
 * @param oRecordLen      Output length of the merged record
 * @return 0 if successful, negative number for error code
 */
int mergeFeatureRecord(int templateFormat, byte[] iFeatureList, int iFeatureNum, byte[]
oFeatureRecord, int[] oRecordLen);

```

2.34 Compare Templates with Multiple Features and Score

```

/**
 * Compares a fingerprint template against multiple feature records and returns matching
 * scores.
 *
 * @param templateFormat  Template format (same codes as above)
 * @param pTemplate       Fingerprint template
 * @param pFeature        Feature records to compare
 * @param oScore          Output score array (0~100)
 * @param pFgTemplateP    Optional pointer to feature template (can be null)
 * @param pFgFeatureP     Optional pointer to feature list (can be null)
 * @return 0 if successful, negative number for error code
 */
int compareTemplatesMultipleFeatureWithScore(int templateFormat, byte[] pTemplate, byte[]
pFeature, int[] oScore, int[] pFgTemplateP, int[] pFgFeatureP);

```